16.2 Auth Profile 与凭据管理

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~400,000 tokens，输出 ~32,000 tokens（本章合计）

LLM API 调用需要认证凭据。OpenClaw 通过 **Auth Profile** 机制统一管理所有提供者的认证信息——包括 API Key、OAuth Token、和特殊的 Copilot Token。同一个提供者可以配置多个 Profile 提供冗余凭据，实现自动轮换和故障隔离。

16.2.1 Auth Profile 机制（`src/agents/auth-profiles.ts`）

Auth Profile 子系统的代码组织在 `src/agents/auth-profiles/` 目录下，通过 `auth-profiles.ts` 聚合导出：

```
src/agents/auth-profiles/
    ├── constants.ts   ← 特殊 Profile ID 常量
    ├── display.ts     ← Profile 显示标签
    ├── doctor.ts      ← 诊断提示
    ├── oauth.ts       ← OAuth Token 解析
    ├── order.ts       ← Profile 排序
    ├── paths.ts       ← 存储路径
    ├── profiles.ts    ← Profile CRUD 操作
    ├── repair.ts      ← Profile ID 修复
    ├── store.ts       ← 持久化存储
    ├── types.ts       ← 类型定义
    └── usage.ts       ← 使用统计与冷却
```

存储结构

Auth Profile 存储在 Agent 目录下的 `auth-profiles.json` 文件中。`AuthProfileStore` 类型定义了存储结构：

```typescript
// src/agents/auth-profiles/types.ts（概念模型）
export type AuthProfileStore = {
  profiles: Record<string, AuthProfileCredential>;
  usageStats?: Record<string, ProfileUsageStats>;
  profileOrder?: Record<string, string[]>;  // 提供者 → Profile ID 列表
};
```

每个 Profile 包含凭据和元数据：

```typescript
export type AuthProfileCredential =
  | ApiKeyCredential
  | OAuthCredential
  | TokenCredential;

export type ApiKeyCredential = {
  type: "api-key";
  provider: string;
  apiKey: string;
  label?: string;
};

export type OAuthCredential = {
  type: "oauth";
  provider: string;
  accessToken: string;
  refreshToken?: string;
  expiresAt?: number;
  label?: string;
};
```

16.2.2 OAuth 认证流程

OpenClaw 支持多个提供者的 OAuth 认证，允许用户使用自己的订阅账号（如 Claude Pro/Max、ChatGPT Plus）而非直接的 API Key。

OAuth 流程的基本步骤：

1. **发起授权**——用户通过 `openclaw auth` 命令启动 OAuth 流程
2. **浏览器授权**——系统打开浏览器，用户在提供者网站上登录并授权
3. **回调接收**——本地 HTTP 服务器接收回调，获取 authorization code

4. **Token 交换**——用 code 交换 access token 和 refresh token
5. **存储 Profile**——将 Token 存储为一个 Auth Profile

每次 LLM 调用时，`resolveApiKeyForProfile` 函数检查 Token 是否过期，必要时使用 refresh token 自动刷新。

16.2.3 API Key 认证

API Key 是最简单的认证方式。用户通过配置文件或 `openclaw auth` 命令设置 API Key：

```
# 配置示例（概念）
agents:
  defaults:
    model:
      primary: "anthropic/claude-sonnet-4-20250514"
```

API Key 可以从多个来源获取：

- Auth Profile Store 中的 `api-key` 类型 Profile
- 环境变量（如 `ANTHROPIC_API_KEY`）
- 系统 Keychain（macOS）

获取优先级为：**Profile Store > 环境变量 > Keychain**。

16.2.4 Profile 轮换与冷却（Cooldown）

当一个 Auth Profile 因为速率限制或其他错误而失败时，系统不会立即放弃，而是将该 Profile 标记为**冷却**状态，然后尝试下一个可用的 Profile。

冷却判断

```
// src/agents/auth-profiles/usage.ts
export function isProfileInCooldown(store: AuthProfileStore, profileId: string): boolea
  const stats = store.usageStats?.[profileId];
  if (!stats) return false;
  const unusableUntil = resolveProfileUnusableUntil(stats);
  return unusableUntil ? Date.now() < unusableUntil : false;
}
```

一个 Profile 处于冷却状态意味着它的 `cooldownUntil` 或 `disabledUntil` 时间戳大于当前时间。

冷却时间计算

冷却时间采用**指数退避**（Exponential Backoff）策略：

```
// src/agents/auth-profiles/usage.ts
export function calculateAuthProfileCooldownMs(errorCount: number): number {
  const normalized = Math.max(1, errorCount);
  return Math.min(
    60 * 60 * 1000,              // 上限：1 小时
    60 * 1000 * 5 ** Math.min(normalized - 1, 3),
  );
}
```

连续失败次数	冷却时间
1	1 分钟
2	5 分钟
3	25 分钟
4+	60 分钟（上限）

> **衍生解释**：**指数退避**是分布式系统中的经典策略。当一个操作失败后，等待时间按指数增长（而非线性增长），以避免在问题未解决时持续发送请求（称为"惊群效应"）。OpenClaw 使用 5 的幂次（5^0=1, 5^1=5, 5^2=25），比常见的 2 的幂次更激进，因为 API 速率限制通常需要较长的恢复时间。

失败标记

```
// src/agents/auth-profiles/usage.ts（简化版）
export async function markAuthProfileFailure(params: {
  store: AuthProfileStore;
  profileId: string;
  reason: AuthProfileFailureReason;
  cfg?: OpenClawConfig;
  agentDir?: string;
}): Promise<void> {
  const stats = store.usageStats?.[profileId] ?? {};
  const errorCount = (stats.errorCount ?? 0) + 1;
  const cooldownMs = calculateAuthProfileCooldownMs(errorCount);

  store.usageStats[profileId] = {
    ...stats,
    errorCount,
    lastError: Date.now(),
    lastErrorReason: params.reason,
    cooldownUntil: Date.now() + cooldownMs,
  };
  // 持久化到磁盘
  saveAuthProfileStore(store, params.agentDir);
}
```

成功重置

当 Profile 被成功使用后，错误计数器重置：

```
// src/agents/auth-profiles/usage.ts
export async function markAuthProfileUsed(params: {
  store: AuthProfileStore;
  profileId: string;
  agentDir?: string;
}): Promise<void> {
  store.usageStats[profileId] = {
    lastUsed: Date.now(),
    errorCount: 0,           // 重置错误计数
    cooldownUntil: undefined, // 清除冷却
    disabledUntil: undefined,
    disabledReason: undefined,
  };
}
```

这意味着一次成功的使用会完全清除该 Profile 的“前科”——即使之前连续失败了多次。

Profile 排序

系统通过 `resolveAuthProfileOrder` 确定 Profile 的尝试顺序。优先使用配置中指定的顺序，其次按最近成功使用的时间排序。冷却中的 Profile 不会从排序中移除（仍可能被选中后跳过），但在实际轮换时会被 `isProfileInCooldown` 过滤。

16.2.5 GitHub Copilot Token 认证

GitHub Copilot 使用了一种特殊的认证流程——需要先用 GitHub Personal Access Token（PAT）换取一个短期的 Copilot API Token：

```
// src/agents/pi-embedded-runner/run.ts（简化版）
if (model.provider === "github-copilot") {
  const { resolveCopilotApiToken } = await import("../../providers/github-copilot-token
  const copilotToken = await resolveCopilotApiToken({
    githubToken: apiKeyInfo.apiKey,  // PAT
  });
  authStorage.setRuntimeApiKey(model.provider, copilotToken.token);  // 短期 Token
}
```

GitHub Copilot 的认证链：

```
GitHub PAT（长期）
    │
    ├── POST https://api.github.com/copilot_internal/v2/token
    │
    ▼
Copilot API Token（短期，约 30 分钟）
    │
    ├── 调用 Copilot Chat API
    │
    ▼
LLM 响应
```

`resolveCopilotApiToken` 函数处理了 Token 的缓存和自动刷新，避免每次 LLM 调用都重新获取 Token。

本节小结

1. **Auth Profile** 是 OpenClaw 的统一凭据管理抽象，支持 API Key、OAuth Token 和特殊 Token 三种类型。
2. **OAuth 认证**允许用户使用订阅账号（Claude Pro/Max、ChatGPT Plus）而非直接的 API Key，系统自动处理 Token 刷新。
3. **Profile 轮换**在凭据失败时自动切换到下一个可用 Profile，**指数退避冷却**避免反复使用已知失败的凭据。
4. 冷却时间从 1 分钟到 60 分钟递增，成功使用一次即完全重置冷却状态。
5. **GitHub Copilot** 使用二阶 Token 交换机制，PAT 换取短期 API Token，由专用函数处理缓存和刷新。